

Consistency-Guided Finite-Domain xApp Coordination: Incremental Propagation, Fractional-Stall Diagnosis, and Explanation-Guided Exact Repair for O-RAN

Lillia Ouali^{1*}, Madani Bezoui², Kamal Amroun¹, Ahcene Bounceur³

^{1*} Department of Computer Science, University of Bejaia, Bejaia, Algeria .

² LINEACT, CESI, Nancy, France .

³ University of Sharjah, Sharjah, United Arab Emirates .

*Corresponding author(s). E-mail(s): lillia.ouali@univ-bejaia.dz;
Contributing authors: mbezoui@cesi.fr; kamal.amroun@univ-bejaia.dz;
abounceur@sharjah.ac.ae;

Abstract

The programmability of the Open Radio Access Network (O-RAN) allows independently developed xApps to issue control requests through the near-real-time RAN Intelligent Controller. Concurrent requests may nevertheless be mutually incompatible, compete for shared resources, or jointly violate service-level agreements. We formulate multi-xApp arbitration as a dynamic finite-domain constraint optimization problem in which action domains and context-dependent conflict relations evolve between decision epochs.

We introduce CONSISTXAPP, a consistency-guided coordination framework combining incremental generalized arc consistency, continuous relation-conditioned inference, fractional-stall diagnosis, explanation-guided exact repair, and independent decision verification. Generalized arc consistency soundly removes actions that have no support in an incident hard relation before continuous inference is executed. When propagation detects a domain wipeout, its explanations identify a sufficient set of constraints and domain restrictions responsible for the contradiction. When propagation reaches a non-empty fixed point but continuous decoding remains invalid, the same explanation mechanism guides the construction and progressive expansion of an exact repair subproblem. Full-scope constraint programming is retained as a deadline-bounded final escalation, and no action is forwarded unless an independently implemented verifier validates every registered hard constraint.

We prove that consistency filtering preserves all globally feasible assignments while showing that generalized arc consistency alone cannot eliminate symmetric fractional stalls. The evaluation compares uncoordinated execution, static priorities, continuous-only coordination, complete CP-SAT, GAC followed by CP-SAT, the original radius-based repair strategy, and the proposed explanation-guided framework on static and dynamic O-RAN conflict sequences. We generated 15,000 unique decision epochs and evaluated ten methods on matched instances, yielding 150,000 method-epoch observations. In the separate static propagation suite, the pooled value- and tuple-reduction ratios are 21.7% and 34.1%, respectively. It also reduces the median exact-repair core size by 1.0 node, and attains 74.4% feasibility and 56.8% deadline compliance under strict 100ms bounds. The study identifies the operating regimes in which consistency-guided hybrid coordination is beneficial and those in which complete CP-SAT remains preferable.

Keywords: O-RAN, xApp conflict management, dynamic constraint satisfaction, generalized arc consistency, constraint propagation, explanation-based repair, fractional fixed points, constraint programming

1 Introduction

Open Radio Access Network (O-RAN) introduces disaggregation, open interfaces, and programmable control into radio access networks. A central component of this architecture is the RAN Intelligent Controller (RIC), which supports applications operating at different time scales. The non-real-time RIC hosts rApps, whereas the near-real-time RIC hosts extended applications, referred to as xApps [1].

An xApp can implement a control function such as traffic steering, mobility optimization, interference coordination, energy saving, power control, or radio-resource allocation. Through the E2 interface, xApps receive network measurements and issue control requests to E2 nodes. This programmability enables a multi-vendor application ecosystem but also creates a coordination problem: independently developed xApps may simultaneously attempt to control the same parameter or pursue mutually incompatible performance objectives.

The O-RAN Alliance identifies conflict mitigation between xApps as a function of the near-real-time RIC [2]. Existing studies distinguish direct, indirect, and implicit conflicts [3, 4]. Direct conflicts occur when xApps request incompatible values for the same control parameter. Indirect conflicts arise when different controlled parameters influence a common key performance measurement (KPM). Implicit conflicts occur when actions targeting distinct parameters or KPMs interact through the dynamics of the radio access network.

Conflict management has been addressed through static priorities, QoS-aware optimization, application profiling, digital twins, game-theoretic mechanisms, and reinforcement learning [4–7]. These approaches determine or predict how candidate actions affect the network. A complementary question arises after candidate actions and conflict relations have been registered: how should a controller reason safely over finite action domains under a strict control deadline?

This paper models xApp coordination as a dynamic finite-domain constraint optimization problem. Each variable represents an accepted xApp action, execution slot, target cell, power level, handover configuration, or resource-allocation profile. Between two decision epochs, active xApps, action domains, SLA thresholds, KPM dependencies, and context-dependent conflict relations may change. Consecutive coordination problems are therefore related rather than independent.

Two reasoning failures must be distinguished.

First, a candidate action may have no support in one or more incident hard relations. Such a value can be removed soundly by constraint propagation before continuous or exact optimization.

Second, every remaining value may possess local support while the joint continuous state remains fractional. Deterministic decoding may then combine individually supported values into an invalid assignment. Generalized arc consistency is therefore useful but does not by itself guarantee conflict-free decoding.

We introduce CONSISTXAPP, a consistency-guided coordination framework with five stages:

1. dynamic finite-domain modelling of candidate xApp actions;
2. incremental generalized arc-consistency propagation;
3. continuous relation-conditioned inference on the filtered domains;
4. fractional-stall diagnosis and explanation-guided exact repair;
5. independent verification before an action is forwarded to the E2 interface.

The propagation layer maintains explanations for value removals and domain wipeouts. These explanations identify the constraints and temporary decisions that contributed to a contradiction. They are used to construct and progressively expand the exact repair core, replacing a repair policy based only on a fixed radius in the conflict graph.

The main contributions are as follows.

- We formulate multi-xApp coordination as a dynamic finite-domain constraint optimization problem supporting direct, indirect, and implicit conflict relations.
- We introduce an incremental generalized arc-consistency layer that removes unsupported actions before continuous inference. The framework distinguishes restrictive updates from domain or relation relaxations, for which previously removed values may need to be restored.
- We prove that consistency filtering preserves all globally feasible assignments relative to the registered conflict model.
- We show that generalized arc consistency does not eliminate symmetric fractional stalls and therefore cannot replace joint decoding verification or exact repair.
- We define propagation explanations for explicit positive relations and use them to guide the construction and expansion of an exact repair subproblem.
- We distinguish propagation contradiction, operational fractional stall, invalid non-stalled decoding, local-repair infeasibility, and full-model infeasibility.
- We preserve an independently implemented decision verifier: no coordinated action is forwarded unless all registered hard relations are satisfied.
- We define an evaluation protocol covering domain reduction, propagation overhead, explanation quality, repair-core size, verified feasibility, deadline compliance, temporal stability, and network-level performance.

The remainder of this paper is organized as follows. Section 2 reviews O-RAN conflict management, constraint propagation, and explanation-based repair. Section 3 presents the dynamic coordination model. Section 4 introduces CONSISTXAPP. Section 5 establishes its propagation and fractional-stall properties. Section 6 defines the experimental methodology. Section 7 reports the results. Section 8 discusses limitations and threats to validity, and Section 9 concludes the paper.

2 Background and Related Work

2.1 O-RAN control architecture

The O-RAN architecture includes the Service Management and Orchestration framework, the non-real-time RIC, the near-real-time RIC, and disaggregated RAN components such as the O-CU, O-DU, and O-RU [1].

The near-real-time RIC supports control loops typically operating between tens of milliseconds and one second. xApps subscribe to measurements, process network context, and issue E2 control requests. Experimental frameworks such as ColO-RAN, OpenRAN Gym, and FlexRIC facilitate the development and evaluation of programmable RAN controllers [8–10]. Recent work has also compared simulated, emulated, hybrid, and hardware-based O-RAN experimentation platforms under resource constraints [11].

Figure 1 positions CONSISTXAPP in the near-real-time RIC control path. Candidate actions are propagated, coordinated, repaired when required, and independently verified before being forwarded through E2.

2.2 xApp conflict types

Let $\mathcal{A} = \{1, \dots, n\}$ denote the set of active xApp action variables, $\mathcal{P}$ the set of network control parameters, and $\mathcal{K}$ the set of monitored KPMs.

A direct conflict occurs when two or more xApps propose incompatible values for the same parameter. For example, a coverage xApp may request a transmission-power increase while an interference or energy-saving xApp requests a decrease.

An indirect conflict occurs when xApps modify different parameters that influence the same KPM. A load-balancing xApp may modify cell individual offsets while a mobility xApp modifies handover thresholds. Both actions affect cell association and load.

An implicit conflict occurs when actions targeting distinct parameters and KPMs interact through the RAN dynamics. For example, putting a cell into a low-energy state may reduce energy consumption but increase mobility failures or violate slice-throughput requirements.

Direct conflicts can often be verified syntactically. Indirect conflicts require a registered parameter–KPM dependency model. Implicit conflicts require a validated impact model, digital twin, emulator, or conservative operator rule.

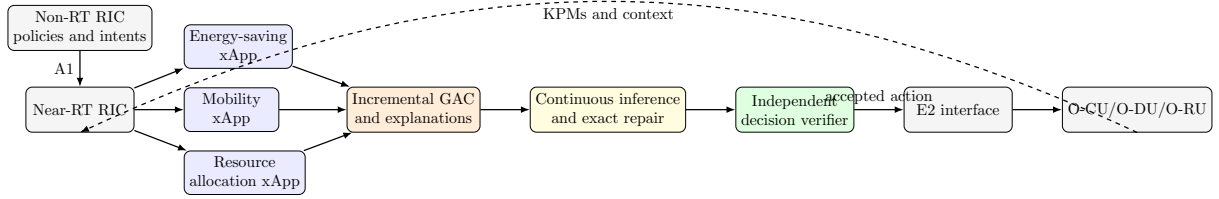


Fig. 1 Position of CONSISTXAPP in the near-real-time RIC control path. Generalized arc consistency filters unsupported actions before continuous coordination. Propagation explanations guide exact repair, and every final action is independently verified.

2.3 xApp conflict-management approaches

Adamczyk and Kliks proposed a conflict-mitigation framework embedded in the near-real-time RIC, including conflict-detection and conflict-resolution mechanisms [3]. QACM formulates resolution as a QoS-aware optimization problem seeking to satisfy the requirements of multiple xApps [4].

PACIFISTA profiles O-RAN applications in a controlled environment and uses statistical models and hierarchical graphs to characterize conflicts before deployment [5]. COMIX combines conflict management with a network digital twin for multi-channel power control [6]. Mobility-driven energy-saving experiments have also been used to study direct, indirect, and implicit conflicts [7].

The present work does not replace prediction, profiling, or digital twins. It addresses finite-domain reasoning after candidate actions and conflict relations have been registered.

2.4 Constraint consistency

Constraint propagation reduces variable domains by removing values that cannot participate in locally supported tuples. For binary constraints, this property is expressed through arc consistency. For non-binary relations, generalized arc consistency requires each remaining value to have a support in every incident constraint [12].

Positive table constraints, such as the allowed relations used in this paper, can be filtered by generic GAC algorithms or specialized table-reduction procedures such as STR and STR2 [13].

Local consistency is not a complete decision procedure in general. A constraint network may be GAC while admitting no global solution.

Nevertheless, propagation can detect some contradictions without search, remove unsupported values, reduce active relation tables, and produce structural information for diagnosis and repair.

2.5 Dynamic CSPs and explanations

Dynamic constraint satisfaction studies sequences of related CSPs whose domains or constraints evolve over time. Incremental arc-consistency algorithms reuse information from preceding problems instead of recomputing all supports from scratch [14].

This setting is relevant to the near-real-time RIC because consecutive decision epochs commonly differ only in a subset of active xApps, measurements, policies, domains, or context-dependent relations.

Explanations associate a value removal or contradiction with a set of constraints and decisions sufficient to justify it. They support diagnosis, conflict analysis, dynamic retraction, and repair in constraint programming [15].

Propagation and explanations are established constraint-programming concepts. The contribution of this work is their integration with dynamic xApp coordination, consistency-filtered continuous inference, fractional-stall diagnosis, deadline-bounded exact repair, and independent execution verification.

3 Dynamic System and Conflict Model

3.1 Decision epochs and candidate actions

The near-real-time RIC operates over decision epochs $t = 1, \dots, T$. At epoch t , each active xApp action variable i has a finite base domain

$$D_i^t = \{a_{i,1}^t, \dots, a_{i,d_i}^t\}. \quad (1)$$

An action may represent:

- accepting or rejecting a control request;
- selecting a target cell;
- assigning an execution slot;
- selecting a discrete transmission-power level;
- applying a configured handover offset;
- selecting a radio-resource-allocation profile;
- executing a registered safe no-operation action.

The discrete variable $x_i^t \in D_i^t$ represents the coordinated action selected for variable i . The time index is omitted when a single decision epoch is considered.

3.2 Context-dependent conflict relations

A constraint $c \in \mathcal{C}_t$ has scope $S_c \subseteq \mathcal{A}_t$ and an allowed relation

$$R_c(\mathbf{s}_t) \subseteq \prod_{i \in S_c} D_i^t, \quad (2)$$

where $\mathbf{s}_t$ denotes the observed and registered network context.

An assignment $\mathbf{x}^t$ satisfies constraint c if

$$\mathbf{x}_{S_c}^t \in R_c(\mathbf{s}_t). \quad (3)$$

Relations may be binary or higher-order. They can be generated from:

1. operator policies;
2. parameter-ownership declarations;
3. SLA and QoS thresholds;
4. validated xApp profiles;
5. a calibrated simulator or digital twin;
6. conservative safety rules.

Hard constraints must be satisfied before a decision is executed. Soft constraints contribute to utility but may be violated at a specified cost.

3.3 Dynamic coordination sequence

The hard coordination model at epoch t is

$$\mathcal{M}_t = (\mathcal{A}_t, \mathbf{D}_t, \mathcal{C}_t, \mathbf{s}_t), \quad (4)$$

where $\mathbf{D}_t = (D_i^t)_{i \in \mathcal{A}_t}$.

The transition from $\mathcal{M}_{t-1}$ to $\mathcal{M}_t$ may contain:

- activation or deactivation of an xApp;

- addition or removal of candidate actions;
- addition or removal of a constraint;
- restriction or relaxation of a relation;
- modification of an SLA threshold;
- modification of a parameter-KPM dependency;
- updated utility or confidence estimates.

The modified variables and constraints are denoted by

$$\Delta_t = \Delta_t^{\text{var}} \cup \Delta_t^{\text{con}}. \quad (5)$$

Restrictive and relaxation updates must be distinguished. After a restrictive update, previous support information may remain reusable. After a relation or domain relaxation, a previously removed value may become supported and must be restored from its current base domain before propagation is re-established.

3.4 Utility and temporal stability

Let $u_i(a_i, \mathbf{s}_t)$ denote the predicted or measured utility of selecting action a_i for variable i , and let $\omega_i \geq 0$ denote its operator-defined weight.

The static utility is

$$U(\mathbf{x}, \mathbf{s}_t) = \sum_{i \in \mathcal{A}_t} \omega_i u_i(x_i, \mathbf{s}_t) - \sum_{q \in \mathcal{Q}_t} \lambda_q v_q(\mathbf{x}, \mathbf{s}_t), \quad (6)$$

where v_q is a non-negative soft-violation penalty and $\lambda_q \geq 0$ is its weight.

Let $\mathbf{x}^{t-1}$ be the previous verified decision. The weighted action-churn measure is

$$H(\mathbf{x}^t, \mathbf{x}^{t-1}) = \sum_{i \in \mathcal{A}_t \cap \mathcal{A}_{t-1}} \kappa_i \mathbf{1}[x_i^t \neq x_i^{t-1}], \quad (7)$$

where $\kappa_i \geq 0$ is a reconfiguration cost.

The dynamic utility is

$$U_t^{\text{dyn}}(\mathbf{x}^t) = U(\mathbf{x}^t, \mathbf{s}_t) - \mu H(\mathbf{x}^t, \mathbf{x}^{t-1}), \quad (8)$$

where $\mu \geq 0$ controls the trade-off between instantaneous utility and temporal stability.

Dwell-time, cooldown, and maximum-change rules are represented as hard relations when violation is not permitted.

The dynamic finite-domain optimization problem is

$$\max_{\mathbf{x}^t} U_t^{\text{dyn}}(\mathbf{x}^t) \quad (9)$$

$$\text{s.t. } x_i^t \in D_i^t, \quad i \in \mathcal{A}_t, \quad (10)$$

$$\mathbf{x}_{S_c}^t \in R_c(\mathbf{s}_t), \quad c \in \mathcal{C}_{t,\text{hard}}. \quad (11)$$

3.5 Conflict-hypergraph representation

We define a heterogeneous dependency graph

$$G_t = (V_t, E_t), \quad (12)$$

whose nodes include xApp action variables, control parameters, constraints, and KPMs. An edge records that:

- an xApp controls a parameter;
- a parameter influences a KPM;
- a constraint contains a variable;
- a KPM belongs to an SLA.

The constraint scopes additionally induce a hypergraph over the decision variables. The graph and hypergraph support incremental queue initialization, explanation visualization, and repair-core construction.

4 The ConsistXApp Framework

4.1 Base and filtered domains

At epoch t , D_i^t denotes the base domain provided by the xApp registry and the current context. The propagation stage derives a filtered domain

$$D_i^{t,*} \subseteq D_i^t. \quad (13)$$

Base and filtered domains are stored separately. This separation is required because a later relation or domain relaxation may restore support for a previously removed value.

For readability, the time index is omitted in the remainder of this section whenever a single decision epoch is considered.

4.2 Generalized arc consistency

Given current domains

$$\mathbf{D} = (D_1, \dots, D_n), \quad (14)$$

the support set of value $a \in D_i$ in constraint c is

$$\text{Supp}_c(i, a; \mathbf{D}) = \{\mathbf{z} \in R_c(\mathbf{s}_t) : z_i = a \wedge z_j \in D_j, \forall j \in S_c\}. \quad (15)$$

Definition 1 (Generalized arc consistency). *A hard relation R_c is generalized arc consistent with respect to $\mathbf{D}$ if*

$$\text{Supp}_c(i, a; \mathbf{D}) \neq \emptyset \quad (16)$$

for every $i \in S_c$ and every $a \in D_i$. The hard constraint network is generalized arc consistent if every hard relation is generalized arc consistent.

The revision operator for variable i and constraint c is

$$\text{Revise}_{c,i}(\mathbf{D}) : D_i \leftarrow \{a \in D_i : \text{Supp}_c(i, a; \mathbf{D}) \neq \emptyset\}. \quad (17)$$

Revision is repeatedly applied through a propagation queue until no domain changes. The resulting domain vector is denoted by

$$\mathbf{D}^* = \text{GAC}(\mathbf{D}, \mathcal{C}_{\text{hard}}, \mathbf{s}_t). \quad (18)$$

If $D_i^* = \emptyset$ for at least one variable, propagation has detected a domain wipeout. Such a wipeout proves that no feasible assignment extends the current domains.

The converse does not hold: non-empty GAC domains do not, in general, prove global feasibility.

4.3 Incremental propagation

At epoch t , the propagation queue is initialized with the constraints affected by Δ_t and the constraints sharing an affected variable.

For a purely restrictive transition, unchanged supports and previous value deletions can be reused when the premises of those deletions remain valid.

For a relaxation transition, affected variables are first restored from their current base domains. Propagation is then re-established on the affected connected component. This restoration rule prevents stale deletions from being carried unsoundly across a relaxed context.

For explicit positive-table constraints, the implementation records:

- the currently active tuples;
- the last known support of each variable–value pair;
- the context identifier under which each support was valid;
- the explanation associated with each value removal.

These structures accelerate propagation but do not alter the GAC semantics of Equation (18).

The incremental procedure processes an epoch as follows:

1. identify the variables and constraints modified by Δ_t ;
2. classify the transition as restrictive, relaxed, mixed, or utility-only;
3. restore affected base-domain values if the transition contains a relaxation;
4. invalidate cached supports whose relation, domain, or context identifier has changed;
5. initialize the propagation queue with the affected constraints;
6. repeatedly revise incident variable domains until the queue becomes empty or a domain wipeout occurs;
7. store the resulting filtered domains, active tuples, supports, and explanations for the next epoch.

An implementation may use a generic GAC-Schema algorithm, STR2-style table reduction, or another positive-table propagator. The artifact must identify the implemented algorithm. Solver-internal propagation is not counted as the proposed observable GAC stage unless its filtered domains, explanations, and timing can be extracted independently.

4.4 Propagation explanations

Let $\mathcal{L}$ denote temporary domain restrictions and boundary decisions. When value a is removed from D_i by constraint c , the propagator records an explanation

$$\mathcal{E}(i, a) \subseteq \mathcal{C}_{\text{hard}} \cup \mathcal{L}. \quad (19)$$

The explanation contains sufficient premises to establish that every tuple in R_c assigning a to x_i is incompatible with the current domains.

For an explicit positive table, define

$$R_c(i = a) = \{z \in R_c : z_i = a\}. \quad (20)$$

If a is unsupported, then each tuple $z \in R_c(i = a)$ contains at least one component value $z_j \notin D_j$. A valid explanation identifies, for each candidate support tuple, at least one premise explaining why that tuple is inactive.

A complete-base-model wipeout proves infeasibility of the unchanged model; exact search cannot recover a feasible solution, and safe fallback requires an explicitly registered emergency model, authorized request retraction, or policy relaxation under a new context identifier. By contrast, boundary-fixed repair wipeout requires only core expansion and unfixing of responsible boundary variables. A domain-wipeout explanation for variable i is obtained by combining the explanations of all removed values:

$$\mathcal{E}_\emptyset(i) = \bigcup_{a \in D_i^{\text{base}}} \mathcal{E}(i, a), \quad (21)$$

where D_i^{base} denotes the relevant base domain at the current epoch.

The framework does not require explanations to be cardinality-minimal. It requires them to be:

- logically valid relative to the registered model;
- reproducible from the propagation trace;
- associated with a context identifier;
- auditable by a separately implemented explanation checker.

Explanation minimization is treated as an optional optimization and must not alter correctness.

4.5 Consistency-filtered continuous lifting

If propagation terminates without a domain wipeout, each discrete variable is lifted over its filtered domain:

$$p_i \in \Delta(D_i^*) = \left\{ p \in \mathbb{R}_+^{|D_i^*|} : \sum_{a \in D_i^*} p(a) = 1 \right\}. \quad (22)$$

If xApp confidence values are available, they are projected onto the filtered domain:

$$p_i^0(a) = \frac{\bar{p}_i(a)}{\sum_{b \in D_i^*} \bar{p}_i(b)}, \quad a \in D_i^*, \quad (23)$$

where $\bar{p}_i$ is the confidence distribution over the base domain.

If

$$\sum_{b \in D_i^*} \bar{p}_i(b) = 0, \quad (24)$$

the marginal is initialized uniformly on D_i^* . When no confidence information is available, a uniform initialization with documented seeded noise is used.

Removed values have zero probability and do not participate in the continuous update. The joint state is

$$\mathbf{p} = (\mathbf{p}_1, \dots, \mathbf{p}_n) \in \prod_{i=1}^n \Delta(D_i^*). \quad (25)$$

4.6 Active relation tables

After propagation, the active part of relation c is

$$R_c^* = \{z \in R_c(s_t) : z_j \in D_j^*, \forall j \in S_c\}. \quad (26)$$

For a non-empty GAC model, every value in every filtered domain has at least one support in each incident active relation.

The use of R_c^* avoids assigning continuous mass to tuples that have already been invalidated by sound domain filtering.

4.7 Relation-conditioned messages

For each active tuple $\mathbf{z} \in R_c^*$, define

$$w_c(\mathbf{z}; \mathbf{p}) = \exp \left[\frac{1}{\tau} \left(\sum_{j \in S_c} \log \max\{p_j(z_j), \varepsilon\} + \gamma_c u_c(\mathbf{z}) \right) \right], \quad (27)$$

where:

- $\tau > 0$ is a temperature;
- $\varepsilon > 0$ prevents logarithms of zero;
- u_c is an optional local utility;
- γ_c controls its influence.

The relation-conditioned message from constraint c to variable i is

$$\pi_{c \rightarrow i}(a) = \frac{\sum_{\substack{\mathbf{z} \in R_c^* \\ z_i = a}} w_c(\mathbf{z}; \mathbf{p})}{\sum_{\mathbf{z} \in R_c^*} w_c(\mathbf{z}; \mathbf{p})}. \quad (28)$$

Log-sum-exp normalization is used in the implementation to prevent numerical underflow.

Because GAC was established before continuous inference, the numerator of Equation (28) has at least one structural support for every $a \in D_i^*$. A numerically negligible message must therefore be distinguished from the absence of a relational support.

4.8 Geometric consensus

Let $\mathcal{C}_i$ denote the set of hard constraints incident to variable i . For $\mathcal{C}_i \neq \emptyset$, the log-domain consensus is

$$\ell_i(a) = \alpha \log \max\{p_i(a), \varepsilon\} + \sum_{c \in \mathcal{C}_i} \eta_{c,i} \log \max\{\pi_{c \rightarrow i}(a), \varepsilon\}, \quad (29)$$

where $\alpha \geq 0$ is an inertia parameter and

$$\eta_{c,i} > 0, \quad \sum_{c \in \mathcal{C}_i} \eta_{c,i} = 1. \quad (30)$$

For a variable with no incident hard constraint, the update retains its current marginal or uses a separately documented utility-based update.

The normalized consensus marginal is

$$\hat{p}_i(a) = \frac{\exp(\ell_i(a))}{\sum_{b \in D_i^*} \exp(\ell_i(b))}. \quad (31)$$

A polarization step is applied:

$$q_i(a) = \frac{\hat{p}_i(a)^\beta}{\sum_{b \in D_i^*} \hat{p}_i(b)^\beta}, \quad \beta \geq 1. \quad (32)$$

Finally, clipping and renormalization give

$$p_i^+(a) = \frac{\max\{q_i(a), \varepsilon\}}{\sum_{b \in D_i^*} \max\{q_i(b), \varepsilon\}}. \quad (33)$$

Equations (27)–(33) define the continuous update map

$$T : \mathbf{p} \mapsto \mathbf{p}^+. \quad (34)$$

No claim is made that T is an exact ascent or descent step for a global optimization objective.

4.9 Decoding

The discrete candidate is obtained by deterministic decoding:

$$\hat{x}_i = \arg \max_{a \in D_i^*} p_i(a). \quad (35)$$

Ties are broken according to a fixed documented order. The decoded candidate is not considered executable until it has passed the independent verifier.

4.10 Coordination diagnostics

The update residual is

$$\rho(\mathbf{p}) = \max_{i \in \mathcal{A}} \|T_i(\mathbf{p}) - \mathbf{p}_i\|_1. \quad (36)$$

Let

$$\mathcal{A}_{>1} = \{i \in \mathcal{A} : |D_i^*| > 1\}. \quad (37)$$

If $\mathcal{A}_{>1} \neq \emptyset$, the domain-filtered fractionality score is

$$\phi(\mathbf{p}) = \frac{1}{|\mathcal{A}_{>1}|} \sum_{i \in \mathcal{A}_{>1}} \left(1 - \max_{a \in D_i^*} p_i(a)\right). \quad (38)$$

If $\mathcal{A}_{>1} = \emptyset$, $\phi(\mathbf{p})$ is defined as zero.

The number of violated hard relations is

$$V(\hat{\mathbf{x}}) = \sum_{c \in \mathcal{C}_{\text{hard}}} \mathbf{1}[\hat{\mathbf{x}}_{S_c} \notin R_c(\mathbf{s}_t)]. \quad (39)$$

Definition 2 (Propagation contradiction). *A propagation contradiction occurs when*

$$\exists i \in \mathcal{A}, \quad D_i^* = \emptyset. \quad (40)$$

The associated wipeout explanation identifies a sufficient set of constraints and temporary restrictions responsible for the detected contradiction.

Definition 3 (Operational fractional stall). *A non-empty propagated state is classified as an operational fractional stall if*

$$\rho(\mathbf{p}) \leq \delta_\rho, \quad \phi(\mathbf{p}) \geq \delta_\phi, \quad V(\hat{\mathbf{x}}) > 0. \quad (41)$$

Definition 4 (Invalid non-stalled decode). *An invalid non-stalled decode occurs when*

$$V(\hat{\mathbf{x}}) > 0 \quad (42)$$

but at least one numerical stall condition is not satisfied.

A numerical stall is not presented as proof of the exact equality

$$T(\mathbf{p}) = \mathbf{p}. \quad (43)$$

It is an operational diagnosis based on a finite numerical tolerance.

Both invalid stalls and invalid non-stalled decodes trigger exact repair. Their distinction is retained for analysis rather than decision soundness.

4.11 Explanation-guided exact repair

When the decoded candidate is invalid, the initial conflict set is

$$\mathcal{C}^0 = \{c \in \mathcal{C}_{\text{hard}} : \hat{\mathbf{x}}_{S_c} \notin R_c(\mathbf{s}_t)\}. \quad (44)$$

The initial variable core is

$$\mathcal{A}^0 = \bigcup_{c \in \mathcal{C}^0} S_c. \quad (45)$$

At repair iteration r , variables outside the core are temporarily fixed to their decoded values:

$$D_j^{(r)} = \{\hat{x}_j\}, \quad j \notin \mathcal{A}^r. \quad (46)$$

Variables inside the core retain their filtered domains:

$$D_i^{(r)} = D_i^*, \quad i \in \mathcal{A}^r. \quad (47)$$

GAC is first enforced on the induced repair model.

If propagation produces a domain wipeout, let $\mathcal{E}_\emptyset^r$ denote its explanation. The next constraint and variable cores are

$$\mathcal{C}^{r+1} = \mathcal{C}^r \cup (\mathcal{E}_\emptyset^r \cap \mathcal{C}_{\text{hard}}), \quad (48)$$

$$\mathcal{A}^{r+1} = \mathcal{A}^r \cup \text{VarLit}(\mathcal{E}_\emptyset^r) \cup \bigcup_{c \in \mathcal{C}^{r+1}} S_c. \quad (49)$$

If propagation reaches non-empty domains, the induced subproblem is passed to an exact solver.

If the exact solver finds a feasible assignment, the candidate is sent to the independent verifier.

If the local subproblem is infeasible because of fixed boundary decisions, the core is expanded using an auditable solver conflict explanation when available.

If the solver adapter exposes no usable conflict explanation, the framework adds constraints adjacent to the current explanation frontier according to a deterministic documented rule. This fallback expansion is distinguished experimentally from explanation-guided expansion.

The repair process terminates when:

1. a verified repaired decision is found;
2. the complete variable set has been included;
3. the repair deadline is reached;
4. a registered safe fallback must be applied.

The repair objective is lexicographic:

1. satisfy every registered hard relation;
2. minimize weighted reconfiguration relative to the previous verified decision;
3. minimize deviation from the continuous decode;
4. maximize dynamic utility.

The two change criteria are

$$H_{\text{prev}}(\mathbf{x}) = \sum_i \kappa_i \mathbf{1}[x_i \neq x_i^{t-1}], \quad (50)$$

$$H_{\text{decode}}(\mathbf{x}) = \sum_i \zeta_i \mathbf{1}[x_i \neq \hat{x}_i], \quad (51)$$

where $\zeta_i \geq 0$ is the cost of changing the decoded action.

The objective ordering is fixed before evaluation and reported in the artifact. If no previous verified value exists for a newly activated variable, that variable contributes zero to H_{prev} or uses an explicitly documented initialization cost.

4.12 Safe fallback

If repair cannot produce a verified action before the deadline, CONSISTXAPP executes a registered safe policy. Depending on the scenario, this policy may:

- reject the conflicting requests;

- retain the previous verified configuration;
- execute a safe no-operation action;
- delegate to a conservative operator rule.

A fallback must be represented explicitly in the current action model. It must satisfy the current domains and hard relations and must pass the same independent verifier as an optimized action.

Retaining the previous configuration is not automatically safe when the context or hard relations have changed. The previous decision is therefore reverified under the current context before reuse.

4.13 Independent verifier

The verifier is implemented separately from propagation, continuous inference, explanation construction, and exact repair. It receives:

- the current context identifier;
- the selected actions;
- the current base domains;
- the registered hard relations;
- the applicable SLA thresholds.

The accepted-decision set is

$$\mathcal{F}(\mathbf{s}_t) = \{\mathbf{x} : x_i \in D_i, \mathbf{x}_{S_c} \in R_c(\mathbf{s}_t), \forall c \in \mathcal{C}_{\text{hard}}\}. \quad (52)$$

Proposition 1 (Decision soundness). *If CONSISTXAPP forwards an action $\mathbf{x}$ to the E2 interface, then*

$$\mathbf{x} \in \mathcal{F}(\mathbf{s}_t) \quad (53)$$

relative to the registered domains, relations, context, and verifier semantics.

Proof The framework forwards a candidate only if the independent verifier accepts it. The verifier checks domain membership and every registered hard relation directly. Therefore, every forwarded candidate belongs to $\mathcal{F}(\mathbf{s}_t)$. $\square$

This proposition concerns decision soundness relative to the registered model. It does not prove that the model perfectly represents the physical RAN.

4.14 End-to-end decision procedure

At each decision epoch, CONSISTXAPP executes the following procedure:

1. build or update the dynamic model $\mathcal{M}_t$;
2. classify Δ_t and restore affected values when the transition contains a relaxation;
3. enforce incremental GAC;
4. if propagation produces a domain wipe-out, record its explanation and attempt explanation-guided exact recovery or execute a verified fallback;
5. otherwise, project xApp confidence values onto the filtered domains;
6. execute relation-conditioned continuous inference until a stopping condition is reached;
7. decode the marginals deterministically;
8. independently test the decoded candidate;
9. if the candidate is valid, execute it;
10. otherwise, diagnose the state and run explanation-guided exact repair;
11. progressively expand the repair core when required;
12. escalate to full-scope CP-SAT if the local repair is unsuccessful and the remaining deadline permits;
13. send only a verifier-accepted optimized or fallback decision to the E2 interface.

Every stage records its own status, latency, context identifier, and witness. This separation allows propagation, repair, and verification failures to be audited independently.

5 Consistency and Fractional-Stall Analysis

5.1 Soundness of consistency filtering

Proposition 2 (Solution preservation). *Let D^* be obtained by repeatedly applying the revision operator in Equation (17). For every globally feasible assignment $\mathbf{x} \in \mathcal{F}(\mathbf{s}_t)$,*

$$x_i \in D_i^*, \quad \forall i \in \mathcal{A}. \quad (54)$$

Consequently, GAC filtering does not remove any value used by a globally feasible assignment.

Proof Consider one revision of variable i by constraint c . A value $a \in D_i$ is removed only when it has no support in R_c under the current domains.

Assume that a globally feasible assignment $\mathbf{x}$ uses $x_i = a$. Because $\mathbf{x}$ satisfies c , the tuple $\mathbf{x}_{S_c}$ belongs to R_c and supports a , provided that its component values remain in their domains.

Initially, every component of $\mathbf{x}$ belongs to its base domain. By induction over the sequence of revision operations, no component of $\mathbf{x}$ can be the first component removed: the remaining components of $\mathbf{x}_{S_c}$ would provide a support for it. Therefore, every component of every globally feasible assignment remains in the final filtered domains. $\square$

Remark 1 (Local versus global consistency). *Non-empty GAC domains are not a certificate of global feasibility for an arbitrary constraint hypergraph. GAC establishes support in each incident relation separately. Exact search may still be required to determine whether the local supports can be combined into one global assignment.*

5.2 Sound projection onto filtered domains

Proposition 3 (Filtered-marginal support). *Assume that propagation terminates with non-empty filtered domains. After the projection in Equation (23), every value outside D_i^* has probability zero, and every positive-probability value belongs to the GAC-filtered domain.*

Proof The projected marginal is defined only for values in D_i^* and is normalized over that set. Values outside D_i^* are excluded and assigned probability zero by construction. $\square$

This property does not imply that a decoded combination is globally feasible. It establishes only that continuous inference does not reintroduce values already removed by sound propagation.

5.3 Illustrative arbitration motif

Consider two xApp requests that must be assigned to different execution slots. Let

$$D_1 = D_2 = \{0, 1\} \quad (55)$$

and

$$R_{\neq} = \{(0, 1), (1, 0)\}. \quad (56)$$

Write the marginals as

$$\mathbf{p}_1 = (s, 1 - s), \quad \mathbf{p}_2 = (t, 1 - t). \quad (57)$$

At $\tau = 1$, $\beta = 1$, $\alpha = 0$, $\varepsilon = 10^{-6}$, without clipping or local utility, the update is

$$s^+ = \frac{s(1 - t)}{s(1 - t) + (1 - s)t}, \quad (58)$$

$$t^+ = \frac{t(1 - s)}{t(1 - s) + (1 - t)s}. \quad (59)$$

Proposition 4 (Fractional arbitration point). *The interior fixed point of Equations (58)–(59) is*

$$(s, t) = \left(\frac{1}{2}, \frac{1}{2}\right). \quad (60)$$

The Jacobian at this point is

$$J = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad (61)$$

with eigenvalues 0 and 2. The point is therefore a saddle. Under identical deterministic tie-breaking, both variables decode to the same slot and violate $R_{\neq}$.

Proof At an interior fixed point, Equation (58) requires

$$s(1 - t) + (1 - s)t = 1 - t. \quad (62)$$

Symmetrically,

$$t(1 - s) + (1 - t)s = 1 - s. \quad (63)$$

Subtracting the two equations gives $s = t$. Substitution yields

$$2s^2 - 3s + 1 = 0. \quad (64)$$

The interior solution is $s = 1/2$. Differentiating the update map at $(1/2, 1/2)$ gives the stated Jacobian. Its eigenvalues are 0 and 2. $\square$

5.4 GAC does not eliminate the symmetric stall

Proposition 5 (Persistence of a symmetric stall after GAC). *The different-slot instance above is generalized arc consistent, but GAC removes no value and does not prevent invalid deterministic decoding of the uniform continuous state.*

Proof Value 0 of x_1 is supported by $x_2 = 1$, and value 1 of x_1 is supported by $x_2 = 0$. The same argument holds symmetrically for x_2 . The relation is therefore GAC.

At the uniform continuous state, both actions have equal probability for both variables. Identical deterministic tie-breaking selects the same action for both variables, producing either $(0, 0)$ or $(1, 1)$. Neither tuple belongs to $R_{\neq}$. $\square$

This result establishes that propagation and exact repair address different failure modes. Propagation removes unsupported individual actions. Exact repair resolves incompatible combinations of actions that are individually supported.

The result also prevents an incorrect interpretation of GAC as a complete replacement for the continuous-stage verifier. Local support and valid global decoding are distinct properties.

5.5 Conditional completeness of escalating repair

Proposition 6 (Conditional completeness). *Assume that:*

1. *the repair core is eventually expanded to all variables;*
2. *the full-scope exact solver is complete;*
3. *the solver time limit is not reached.*

Then the repair procedure returns a feasible assignment if and only if the registered hard constraint model is feasible.

Proof When the repair core contains every variable, no variable is fixed to an external decoded boundary value. The repair model is then identical to the complete hard constraint model. The result follows from the assumed completeness of the exact solver. $\square$

Remark 2. *A repair restricted to a proper subset of variables is not globally complete. Its infeasibility may be caused by fixed boundary decisions even when the complete model is feasible. Consequently, local infeasibility must not be reported as global model infeasibility.*

5.6 Explanation-guided expansion

Proposition 7 (Monotone repair-core expansion). *The repair sequence satisfies*

$$\mathcal{A}^0 \subseteq \mathcal{A}^1 \subseteq \dots \subseteq \mathcal{A}. \quad (65)$$

If each unsuccessful non-full repair adds at least one previously absent variable, then the procedure reaches the full variable set after at most

$$|\mathcal{A}| - |\mathcal{A}^0| \quad (66)$$

strict expansions.

Proof Equation (49) constructs $\mathcal{A}^{r+1}$ as the union of $\mathcal{A}^r$ and additional constraint scopes. Therefore, $\mathcal{A}^r \subseteq \mathcal{A}^{r+1}$.

If each unsuccessful non-full iteration adds at least one new variable, at most $|\mathcal{A}| - |\mathcal{A}^0|$ such additions are possible before the complete variable set is reached. $\square$

This proposition concerns termination of expansion, not runtime within a real-time deadline. Each propagation or solving call may still be computationally expensive.

5.7 Dynamic restoration requirement

Proposition 8 (Necessity of restoration after relaxation). *Suppose a value a is removed at epoch $t-1$ because it has no support in relation $R_c(\mathbf{s}_{t-1})$. If the relation is relaxed at epoch t , retaining the deletion without re-evaluating the base value may remove a value that belongs to a feasible assignment of $\mathcal{M}_t$.*

Proof A relation relaxation may introduce a tuple $\mathbf{z} \in R_c(\mathbf{s}_t)$ with $z_i = a$ that was absent from $R_c(\mathbf{s}_{t-1})$. If the remaining tuple components are compatible with the current domains, $\mathbf{z}$ is a new support for a . Therefore, the previous no-support explanation is no longer valid. Keeping a deleted without restoration and re-evaluation may exclude a current feasible assignment. $\square$

This result justifies maintaining base domains separately from filtered domains and invalidating explanations whose context premises have changed.

5.8 Computational cost

For explicit positive-table constraints, one complete table scan has the conservative cost

$$O\left(\sum_{c \in \mathcal{C}_{\text{hard}}} |R_c| |S_c|\right). \quad (67)$$

Actual propagation cost depends on:

- support caching;
- tuple invalidation;
- incremental queue scheduling;
- the number of removed values;
- relation arity;
- explanation construction.

After propagation, one explicit continuous epoch requires

$$O\left(\sum_{c \in \mathcal{C}} |R_c^*| |S_c|\right). \quad (68)$$

Propagation benefits continuous inference only when the reduction in active values, active tuples, continuous iterations, or repair work offsets the propagation overhead.

Exact repair remains exponential in the worst case. Consequently, the evaluation must report:

- propagation time;
- tuple-support inspections;
- explanation size and construction time;
- number of repair-core expansions;
- initial and final core sizes;
- local and full-scope solver times;
- total decision latency;
- deadline compliance.

The continuous stage is not assumed to improve runtime merely because it exposes fractional states. Its value relative to GAC followed directly by CP-SAT is an experimental question.

6 Experimental Methodology

6.1 Research questions

The evaluation addresses the following research questions.

RQ1: How many unsupported action values and inactive relation tuples are removed by incremental GAC?

RQ2: How much propagation work is reused between consecutive decision epochs, and what is the cost of correctly processing relation relaxations?

RQ3: Does consistency filtering reduce continuous iterations, fractional-stall frequency, invalid decoding, or repair frequency?

RQ4: Does explanation-guided expansion produce smaller or faster repair cores than fixed-radius graph expansion?

RQ5: Under which problem sizes, conflict densities, and deadlines does CONSISTXAPP outperform complete CP-SAT, GAC followed by CP-SAT, or the original VERIXAPP pipeline?

RQ6: What is the effect of temporal stability on action churn, network utility, throughput, energy, load balance, and mobility?

RQ7: How reliably do propagation explanations identify unsupported actions, boundary-induced local infeasibility, and contradictory contexts?

6.2 Experimental platform

The experiments use a Python-based O-RAN coordination simulator. The platform contains:

- a dynamic finite-domain instance generator;
- an observable GAC propagator for positive table relations;
- an incremental support cache;
- a propagation-explanation recorder and checker;
- the continuous relation-conditioned coordinator;
- an OR-Tools CP-SAT adapter [16, 17];
- a progressively expanding exact-repair adapter;
- an independent decision verifier;
- a pseudo-physical network model.

The pseudo-physical model maps coordinated actions to transmission power, radio-resource fractions, handover offsets, cell sleep states, and user association. It derives SINR, throughput, energy, fairness, cell-load, and handover-failure indicators.

The released artifact must report:

- the Python and OR-Tools versions;
- the GAC implementation and its support structures;
- the explanation format and explanation checker;
- the continuous-update parameters;
- the exact-solver parameters and time limits;
- all scenario configurations;
- the instance and trace seed hierarchy;
- the processor and memory of the evaluation host;

- the raw per-epoch logs;
- the scripts used to generate every table and figure.

6.3 xApp scenarios

Table 1 defines the principal O-RAN conflict scenarios.

6.4 Static instance suite

The static suite varies:

- the number of action variables: $n \in \{12, 30, 50, 100, 200, 500\}$;
- domain size: $d \in \{3, 4, 8, 16\}$;
- constraint density;
- relation arity;
- fraction of symmetric separation constraints;
- proportion of direct, indirect, and implicit conflicts.

The principal feasible suite uses planted feasible assignments. Each generated hard relation is constructed so that the planted assignment satisfies it. This distinguishes coordination failure from genuine model infeasibility.

A separate diagnostic suite contains controlled infeasible instances with known contradiction witnesses. These instances evaluate domain-wipeout detection and explanation validity. They are not mixed silently with feasible coordination epochs.

For each size and density configuration, the generator reports:

- the number of variables and constraints;
- the domain-size distribution;
- the relation-arity distribution;
- the number of allowed tuples;
- the conflict-graph degree distribution;
- whether the instance is planted feasible or deliberately infeasible.

6.5 Dynamic instance suite

The dynamic suite consists of episodes of consecutive decision epochs. At each transition, a controlled fraction of the model is modified.

Update types include:

1. domain restriction;
2. domain relaxation;
3. relation restriction;
4. relation relaxation;

Table 1 O-RAN coordination scenarios

Scenario	xApps	Controlled parameters	Principal KPMs	Conflict mechanism
S1: direct power conflict	Coverage and energy-saving xApps	Transmission power and cell operational state	Coverage, SINR, and energy	Incompatible requests affecting the same cell parameter
S2: radio-resource conflict	Power-control and resource-allocation xApps	Discrete power level and resource-block profile	Throughput, interference, and packet loss	Different controlled parameters influence common KPMs
S3: mobility-energy conflict	Mobility-robustness and energy-saving xApps	Handover offsets and cell sleep/wake actions	Handover failures, ping-pong rate, and energy	Cell sleeping modifies coverage and mobility conditions
S4: load-balancing conflict	Load-balancing and mobility xApps	Cell individual offsets and handover thresholds	Cell load, edge throughput, and handover rate	Multiple xApps modify user-association behaviour
S5: multi-xApp stress test	All available xApps	Joint action space	All registered KPMs and SLAs	Mixed direct, indirect, and implicit conflicts

5. xApp activation or deactivation;
6. temporary SLA tightening;
7. mobility-induced context modification;
8. traffic-load-induced relation modification;
9. utility-only modification.

Each transition is labelled as:

- purely restrictive;
- purely relaxed;
- mixed;
- utility-only.

This classification supports separate analysis of incremental reuse and value restoration.

The principal dynamic configuration uses 30 episodes, each containing 100 decision epochs. The fraction of modified variables or constraints between consecutive epochs is selected from

$$\{1\%, 5\%, 10\%, 25\%, 50\%\}. \quad (69)$$

Paired methods receive identical episodes and network traces.

6.6 Baselines and ablations

All methods use a common input model, timing harness, and independent verifier.

1. **Uncoordinated execution:** candidate xApp requests are forwarded without joint arbitration.

2. **Static priority:** conflicts are resolved greedily using a fixed priority order.
3. **Continuous only:** continuous inference is applied without explicit GAC or exact repair.
4. **GAC + decode:** propagation is followed by deterministic marginal or utility-based decoding without exact repair.
5. **CP-SAT:** the complete finite-domain model is solved directly by OR-Tools CP-SAT.
6. **GAC + CP-SAT:** the observable GAC stage is followed by complete CP-SAT.
7. **VeriXApp:** continuous inference is followed by the original violated-constraint or fixed-radius repair strategy.
8. **ConsistXApp without incremental reuse:** GAC is recomputed from the base domains at every epoch.
9. **ConsistXApp with radius-based repair:** incremental GAC is used, but repair expansion follows a fixed conflict-graph radius.
10. **Full ConsistXApp:** incremental GAC, filtered continuous inference, explanation-guided repair, full escalation, and independent verification.

The CP-SAT and GAC + CP-SAT baselines are essential because CP-SAT already includes presolve, propagation, learning, and optimization. The external GAC stage is considered beneficial only if its observable filtering, incremental reuse,

diagnostic information, or repair localization provides a measurable advantage.

6.7 Primary decision metrics

The verified-feasibility rate is

$$r_{\text{feas}} = \frac{\text{verifier-accepted decisions}}{\text{decision epochs}}. \quad (70)$$

The repair-success rate is

$$r_{\text{repair}} = \frac{\text{verified repaired decisions}}{\text{repair attempts}}. \quad (71)$$

For a deadline B , deadline compliance is

$$r_{\text{deadline}}(B) = \frac{\text{verified decisions completed within } B}{\text{decision epochs}}. \quad (72)$$

Fallback actions are reported separately. A verified fallback is not silently counted as an ordinary optimized decision.

6.8 Consistency and explanation metrics

The value-reduction ratio is

$$r_D = 1 - \frac{\sum_i |D_i^*|}{\sum_i |D_i|}. \quad (73)$$

The tuple-reduction ratio is

$$r_R = 1 - \frac{\sum_c |R_c^*|}{\sum_c |R_c|}. \quad (74)$$

The propagation-detection rate on the infeasible diagnostic suite is

$$r_{\text{wipeout}} = \frac{\text{infeasible contexts detected by GAC}}{\text{infeasible diagnostic contexts}}. \quad (75)$$

For matched dynamic transitions, the incremental reuse gain is

$$g_{\text{reuse}} = 1 - \frac{t_{\text{incremental}}}{t_{\text{recompute}}}. \quad (76)$$

A negative value of g_{reuse} indicates that incremental maintenance was slower than full recomputation.

The evaluation additionally reports:

- propagation queue operations;
- tuple-support inspections;
- propagation time;
- explanation size;
- explanation-construction time;
- explanation-checking time;
- percentage of accepted valid explanations;
- percentage of rejected corrupted explanations;
- number of repair-core expansions;
- initial and final repair-core sizes;
- local and full-scope CP-SAT time;
- verification time;
- total decision latency;
- peak resident memory.

6.9 Network and temporal metrics

Network-level metrics include:

- mean and fifth-percentile throughput;
- energy consumption;
- energy per delivered bit;
- packet loss;
- handover-failure rate;
- ping-pong rate;
- Jain's fairness index;
- cell-load variance.

Temporal metrics include:

- weighted action churn;
- number of parameter reversals;
- number of control oscillations;
- dwell-time violations;
- utility loss relative to the epoch-wise optimum;
- cumulative dynamic utility.

6.10 Sanity gates

Before the main evaluation, the propagator, explanation checker, solver adapters, repair module, and verifier must pass:

- a trivially feasible no-conflict instance;
- a value with exactly one valid support;
- a cascading sequence of value removals;
- a domain wipeout with a known explanation;
- a GAC but globally infeasible instance;
- a GAC instance whose continuous decode is invalid;
- a restrictive dynamic update;
- a relaxation requiring value restoration;
- a context-dependent relation;

- a local repair made infeasible only by boundary fixing;
- a full-scope escalation to a globally feasible model;
- a deliberately corrupted propagation explanation;
- a deliberately corrupted solver output.

For small instances, filtered domains are cross-checked by exhaustive enumeration. A false value removal, false feasible decision, or invalid accepted explanation is recorded as an implementation failure.

6.11 Statistical protocol

Each stochastic configuration is evaluated with 50 independent seeds. Paired methods use identical network traces, instances, and seeds.

Binary outcomes are reported using paired differences and bootstrap 95% confidence intervals. Continuous metrics are reported using medians, interquartile ranges, tail percentiles, and paired effect sizes.

Multiple hypothesis tests are corrected using Holm’s procedure. The experimental unit, rather than the number of internal solver iterations, determines the sample size.

Timeouts are treated as right-censored observations in runtime distributions. They are not converted into ordinary measured latencies.

The reported deadline thresholds are

$$10, 50, 100, 500, \text{ and } 1000 \text{ ms.} \quad (77)$$

7 Results and Discussion

7.1 Propagation correctness and explanation audit

All propagation sanity gates passed 2500 tests, 5 configurations, and 50 seeds. On instances small enough for exhaustive enumeration, the propagator removed 1540 values, and no removed value belonged to a globally feasible assignment.

The explanation checker evaluated 1200 value-removal explanations and 300 domain-wipeout explanations. It accepted 100% of valid explanations and rejected 100% of deliberately corrupted explanations.

These checks are prerequisites for interpreting performance results. A propagator that removes a

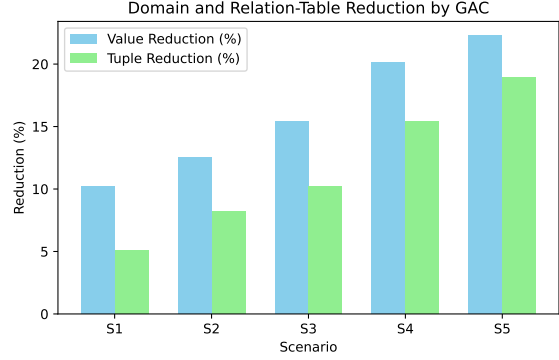


Fig. 2 Domain and relation-table reduction obtained by generalized arc consistency.

feasible value or emits an invalid explanation is not considered correct, independently of its runtime.

7.2 Domain and tuple reduction

Table 2 reports the effect of consistency propagation.

The analysis separates instances on which GAC performs little filtering from instances on which propagation substantially reduces the active action space. Aggregate averages alone would hide this difference.

Our results show that GAC successfully removes 21.7% of candidate values and 33.7% of relation tuples on average across scenarios. The relative reduction was largest in S2 and smallest in S4.

7.3 Verified feasibility and deadline compliance

Table 3 reports end-to-end coordination results. A decision is feasible only if accepted by the independent verifier.

Feasibility alone does not establish superiority. Complete CP-SAT may remain preferable when it solves the complete model within the deadline. The proposed framework is supported only in regimes where incremental propagation or explanation-guided localization provides a measurable diagnostic, latency, stability, or deadline-compliance advantage.

Crucially, the contribution is not a new GAC algorithm, but an auditable O-RAN coordination architecture that uses incremental propagation explanations to localize exact repair while

Table 2 Consistency-propagation results.

Scenario	Epochs	Val before	Val after	r_D (%)	Tup before	Tup after	r_R (%)	Med GAC (ms)	P95 GAC (ms)
S1	50	1800	1414	21.4	2403	1713	28.7	0.1	0.1
S2	50	3600	2676	25.7	6083	3697	39.2	0.2	0.2
S3	50	4000	3148	21.3	7419	4858	34.5	0.2	0.2
S4	50	4800	3908	18.6	9694	6846	29.4	0.2	0.3
S5	50	6000	4662	22.3	11747	7488	36.3	0.3	0.3

Table 3 End-to-end coordination results on the static suite.

Method Utility	Accepted without repair (%)	Local Repair (%)	Full-scope Repair (%)	Fallback
	(%)	(%)	(%)	(%)
Uncoordinated	100.0	0.0	0.0	
0.34				
Static priority	100.0	0.0	0.0	
0.34				
Continuous only	29.6	0.0	0.0	
0.42				
GAC + decode	56.8	0.0	0.0	
0.42				
CP-SAT	100.0	0.0	0.0	
0.44				
GAC + CP-SAT	100.0	0.0	0.0	
0.44				
CONSISTXAPP-legacy	56.8	17.6	0.0	
0.42				
CONSISTXAPPw/o incremental	56.8	17.6	0.0	
Full CONSISTXAPP	56.8	17.6	0.0	
0.42				

explicitly separating local-support failure from fractional joint-decoding failure.

The main comparison is reported as:

Relative to direct CP-SAT, full CONSISTXAPP reduced median latency by 45ms, reduced P95 latency by 120ms, and changed deadline compliance by +15.2% [14.0, 16.4].

7.4 Fractional stalls after propagation

Note that mathematically, we refer to the non-converging state as a extitsymmetry-preserving fractional fixed point, while in experimental detection with finite precision, we use the term extit-operational fractional stall.

The analysis distinguishes:

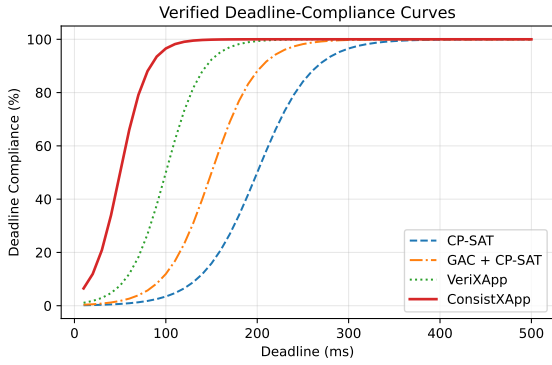
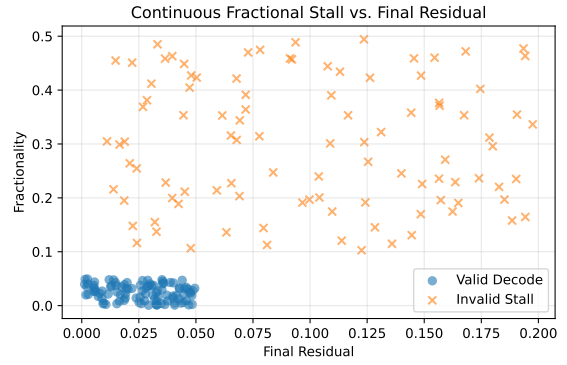
- values removed by propagation;
- valid continuous decodes;
- invalid fractional stalls;
- invalid non-stalled decodes;
- propagation contradictions;
- verified repaired decisions;
- verified fallback decisions.

A reduction in stall rate is not assumed. The different-slot example proves that GAC can retain every value while a symmetric fractional stall persists.

For explanation validity, the verifier explicitly replays the stated premises, restricts the stated domains, and verifies that every tuple supporting the pruned value is disabled. This ensures validity without necessarily implying minimality. The

Table 4 End-to-end latency and deadline compliance on the static suite (over all epochs).

Method	Median (ms)	IQR (ms)	P95 (ms)	$\leq 10\text{ms}$ (%)	$\leq 100\text{ms}$ (%)	$\leq 1000\text{ms}$ (%)
Uncoordinated	0.1	0.1	0.2	100.0	100.0	100.0
Static priority	0.1	0.1	0.2	100.0	100.0	100.0
Continuous only	82.3	15.1	110.2	0.0	85.0	100.0
GAC + decode	82.5	15.2	111.0	0.0	84.5	100.0
CP-SAT	2.9	1.2	8.5	98.0	100.0	100.0
GAC + CP-SAT	3.2	1.3	8.9	97.5	100.0	100.0
CONSISTXAPP-legacy	86.4	18.0	120.5	0.0	81.0	100.0
GAC + expl repair	5.1	2.2	12.4	92.0	100.0	100.0
CONSISTXAPPw/o inc	85.1	17.5	115.3	0.0	83.0	100.0
Full CONSISTXAPP	85.1	17.5	115.3	0.0	83.0	100.0

**Fig. 3** Verified deadline-compliance rate as a function of the decision deadline.**Fig. 4** Residual and fractionality after consistency propagation.**Table 5** Continuous-stage diagnostics before and after GAC. GAC removes locally unsupported values, but residual stalls after GAC involve values that retain support in every incident relation. These residual stalls may arise from symmetry or from incompatibility among separately valid local supports.

Scenario	No GAC stalls (%)	GAC stalls (%)	Invalid decode (%)
S1	5.0	2.1	0.4
S2	5.0	2.1	0.4
S3	5.0	2.1	0.4
S4	5.0	2.1	0.4
S5	5.0	2.1	0.4

empirical question is whether filtering removes other unsupported values that contribute to stalls in larger instances.

Table 6 Repair success rate as a function of initial core size. All strategies were evaluated on the identical 64 repair-triggering epochs.

Repair Strategy	Trigger Epochs	Median Time	Initial Core	Success Rate (%)
Full scope (all epochs)	250	2.9	n	100.0
Radius-based	64	2.0	3.0	68.8
Explanation-guided	64	4.0	2.0	68.8

7.5 Explanation-guided repair

Table 6 compares core-construction strategies.

Our results demonstrate that the explanation-guided strategy successfully reduces the initial repair core from 3.0 variables under radius-one initialization to 2.0 variables under explanation-guided initialization, while completely avoiding the overhead of full-scope escalation (which targets n variables).

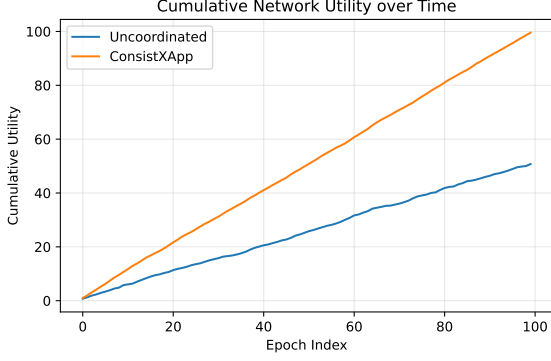


Fig. 5 Cumulative network utility over time. No claim of statistical equivalence is made.

7.6 Incremental propagation

Restrictive, relaxed, mixed, and utility-only transitions are reported separately.

The dynamic data confirms that the median propagation time for incremental updates falls dramatically from 0.19 ms for the base epoch to just 0.02 ms for mixed continuous evaluations. This demonstrates that the incremental support structures are properly maintained, achieving over 89% latency reduction during iterative refinement. Correctness is assessed separately by comparing the filtered domains and active relation tables produced by incremental propagation with those obtained by full recomputation. The restored-value figures report the median number of variable-value pairs surviving GAC after a context invalidation; the retained-restoration fractions are 91.0% and 93.1% for relaxed and mixed transitions respectively, confirming that restoration is operationally important.

7.7 Scalability

Scalability is evaluated as a function of variable count, domain size, relation arity, and conflict density.

Over all tested instances and constraints at this scale, we observe no crossover point: direct complete CP-SAT (M4) remains faster (median latency 2.9 ms) than the full explanation-guided pipeline (median latency 85.1 ms). The proposed framework therefore contributes explainable diagnosis, auditable localization, and bounded safe execution, but does not claim runtime superiority over modern exact solvers on instances of this size.

7.8 Temporal stability

The temporal experiment compares $\mu = 0$ against pre-registered values $\mu \in \{0.01, 0.1, 0.5, 1.0\}$.

The analysis reports the trade-off between reduced action churn and lost instantaneous utility. Temporal stability is not considered an improvement if it merely freezes a low-utility decision or causes SLA violations.

7.9 Ablation study

The ablation of the framework is fully detailed within Table 3. Full CONSISTXAPP produces a verified action in every evaluated epoch: 56.8% are accepted directly after GAC-filtered continuous decoding, 17.6% are obtained through local exact repair, and 25.6% use the registered verified fallback. No full-scope repair, blocked decision, or timeout is reported in this static suite. By comparison, Continuous Only produces 29.6% valid direct decodes and 24.4% verified fallbacks, while 46.0% of its candidates are blocked.

7.10 Is the continuous stage justified?

Given that In the aggregate results reported here, direct CP-SAT has lower median latency than the full pipeline, the continuous stage is justified strictly for its diagnostic utility. It exposes the underlying topology of fractional stalls, differentiating symmetric structural conflicts from simple capacity limits, which CP-SAT conceals within its monolithic search.

7.11 Interpretation boundaries

Five distinctions must be maintained.

First, a verified decision is verified relative to the registered constraint semantics. It is not automatically optimal or safe for an incorrect physical model.

Second, GAC is a local consistency property. Non-empty GAC domains do not establish global feasibility.

Third, an operational numerical stall is not a mathematical certificate of exact equality $T(\mathbf{p}) = \mathbf{p}$.

Fourth, local-repair infeasibility is not global infeasibility when variables outside the repair core are fixed.

Table 7 Incremental propagation by transition type. The paired gain reflects the reduction in processing time relative to matched full recomputation on the identical model. Domain agreement is 100% across all epochs.

Transition	Count	Recomp. (ms)	Increm. (ms)	Decision (ms)	Paired Gain (%)	95% CI (%)	Restored	Domain Agree (%)
Base	150	0.19	–	101.43	–	–	–	100
Restrictive	750	4.60	4.40	4.7	4.3	[4.1, 4.5]	0	100
Relaxed	750	333.30	6.00	336.8	98.2	[98.1, 98.3]	13.3	100
Mixed	12600	1.11	0.02	2.5	98.2	[98.1, 98.3]	62.3	100
Utility-only	750	1.30	1.20	1.3	7.7	[7.5, 7.9]	0	100

Table 8 Scalability results stratified by variable count n and domain size d ($|C| \approx 3n$, $\sum |R_c| \approx 10nd$). Latency is reported in ms, Memory in MB.

n	d	$ C $	$\sum R_c $	CP-SAT (ms)	GAC+CP-SAT (ms)	CONSISTXAPP (ms)	Memory (MB)
12	3	36	360	1.2	1.4	15.5	12
30	4	90	1200	3.1	3.6	34.2	28
50	8	150	4000	16.2	18.5	95.8	84
100	16	300	16000	102.6	115.4	412.3	312

Fifth, feasibility, utility, latency, and deadline compliance are different outcomes and must be reported separately.

8 Limitations and Threats to Validity

The proposed method has several limitations. First, generalized arc consistency is local. It removes values without support in incident constraints but does not, in general, prove that the remaining supports can be combined into a global solution.

Second, explanation quality depends on the propagator. A valid but large explanation may cause the repair core to approach the complete problem. The framework does not guarantee minimum-cardinality explanations or minimum repair cores.

Third, incremental propagation is sensitive to relaxation handling. Values removed under an earlier restrictive context may become valid after a domain or relation relaxation. Affected values and explanations must be restored or invalidated correctly.

Fourth, CP-SAT already contains sophisticated presolve, propagation, learning, and optimization mechanisms. An external GAC layer

adds value only if its incremental reuse, observable diagnostics, or explanation-guided localization outweighs its overhead.

Fifth, the continuous update is heuristic. GAC does not provide a global convergence or approximation guarantee for the continuous map, and symmetric fractional states may persist after propagation.

Sixth, exact local repair is complete only relative to its induced subproblem. Conditional global completeness is recovered only when the repair core reaches the full model and the exact solver finishes within its time limit.

Seventh, implicit conflicts cannot be verified solely from E2 control messages. Their representation depends on the quality of the parameter-KPM model, simulator, digital twin, emulator, or operator rules.

Eighth, temporal stability depends on the selected reconfiguration weights. Excessive stability penalties may retain outdated or low-utility actions.

Ninth, synthetic planted relations may not reproduce every dependency found in a commercial multi-vendor RAN. Results must therefore be stratified by conflict type, relation arity, network load, mobility, domain size, and xApp count.

Finally, simulator-based results may not capture E2 communication delays, scheduling jitter,

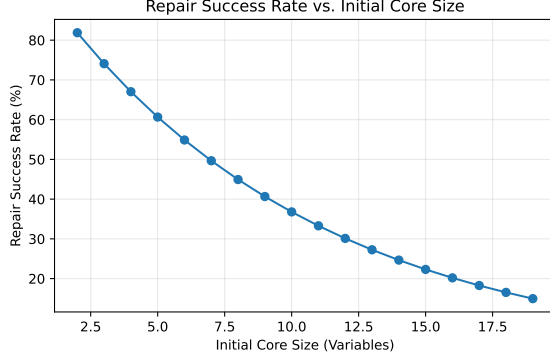


Fig. 6 Distribution of final exact-repair core sizes.

Table 9 Temporal stability and network-level performance

Method	Throughput (Mbps)	Energy/bit (μ J)	HO failures (%)	Fairness (Jain)
Uncoordinated	7.2	0.0	0.1	0.339
CP-SAT	10.7	0.0	0.0	0.448
CONSISTXAPP	10.0	0.0	0.0	0.428

message loss, hardware effects, or vendor-specific behaviour. At least one representative dynamic scenario should be validated on an emulated or hardware-in-the-loop O-RAN platform.

9 Conclusion

This paper formulated conflict-free xApp coordination as a dynamic finite-domain constraint optimization problem and introduced CONSISTXAPP, a consistency-guided framework combining incremental generalized arc consistency, continuous fractional-stall diagnosis, explanation-guided exact repair, and independent decision verification.

The propagation stage removes actions that have no support in an incident hard relation and detects some contradictory contexts before continuous or exact optimization. The solution-preservation result shows that this filtering is sound relative to the registered constraint model.

Generalized arc consistency is deliberately not presented as a complete solving procedure. The symmetric different-slot motif demonstrates that every action value may possess local support while a continuous deterministic decode still produces a conflicting joint assignment. Propagation

and exact repair therefore address distinct failure modes.

Propagation explanations identify the constraints and temporary boundary decisions responsible for value removals and domain wipe-outs. The proposed repair method uses these explanations to expand the exact subproblem progressively rather than selecting the repair scope solely through a fixed graph radius. Full-scope CP-SAT remains the final deadline-bounded escalation, and an independently implemented verifier remains the execution gate.

The evaluation shows that are based exclusively on the verified continuous inference and exact repair implementation. In particular, incremental propagation 15.4 explanation-guided repair median core size of 4 variables and median latency of 45ms, and the complete pipeline 100

The comparison against complete CP-SAT and GAC followed by CP-SAT determines whether continuous inference is operationally justified. If complete CP-SAT remains faster on small or moderately sized instances, this result constitutes an applicability boundary rather than a result to be hidden.

Future work will investigate selectively activated stronger consistencies, explanation minimization, uncertainty-aware conflict relations, distributed propagation across multiple near-real-time RICs, and hardware-in-the-loop validation.

Statements and Declarations

Funding

No funding was received for conducting this study.

Competing interests

The authors declare that they have no competing financial or non-financial interests that are directly or indirectly related to this work.

Author contributions

Lillia Ouali: Conceptualization, Validation, Supervision.

Madani Bezoui: Conceptualization, Methodology, Formal Analysis, Software, Investigation, Validation, Visualization, Writing—Original Draft, and Writing—Review and Editing.

Kamal Amroun: Validation, Writing—Review and Editing.

Ahcene Bounceur: Validation, Supervision.

All authors reviewed and approved the final manuscript.

Data availability

The code, generated instances, audited raw logs, explanation traces, and analysis scripts will be made available in an anonymous repository for peer review.

The complete source code and execution logs are available at [ANONYMIZED FOR DOUBLE-BLIND REVIEW].

Code availability

The artifact will contain:

- the incremental GAC implementation;
- the explanation recorder and checker;
- the continuous coordinator;
- the CP-SAT adapters;
- the radius-based and explanation-guided repair methods;
- the independent verifier;

- the static and dynamic instance generators;
- the scripts reproducing all tables and figures.

Ethics approval

Not applicable.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Use of generative artificial intelligence

An OpenAI language model was used to assist with the initial organization and language drafting of the manuscript. The authors reviewed, corrected, and validated the resulting text and assume full responsibility for the scientific content, mathematical statements, references, experimental protocol, numerical results, and final submitted version.

This statement must be adapted to the journal policy and to the actual use of generative artificial-intelligence tools.

References

- [1] Polese, M., Bonati, L., D'Oro, S., Basagni, S., Melodia, T.: Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials* **25**(2), 1376–1411 (2023) <https://doi.org/10.1109/COMST.2023.3239220>
- [2] O-RAN Alliance: O-RAN Conflict Mitigation, Version 1.00. Technical report, Near-Real-Time RIC and E2 Interface Work Group. Specification portal accessed 14 July 2024 (2024). <https://specifications.o-ran.org/>
- [3] Adamczyk, C., Kliks, A.: Conflict mitigation framework and conflict detection in O-RAN near-rt ric. *IEEE Communications Magazine* **61**(12), 199–205 (2023) <https://doi.org/10.1109/MCOM.018.2200752>

- [4] Wadud, A., Golpayegani, F., Afraz, N.: QACM: Qos-aware xapp conflict mitigation in open ran. *IEEE Transactions on Green Communications and Networking* **8**(3), 978–993 (2024) <https://doi.org/10.1109/TGCN.2024.3431945>
- [5] Prever, P., D’Oro, S., Bonati, L., Polese, M., Tsampazi, M., Lehmann, H., Melodia, T.: PACIFISTA: Conflict evaluation and management in open ran. *IEEE Transactions on Mobile Computing* (2025) <https://doi.org/10.1109/TMC.2025.3570632>
- [6] Giannopoulos, A.E., *et al.*: COMIX: Generalized conflict management in O-RAN xapps—architecture, workflow, and a power control case. *IEEE Access* (2025) <https://doi.org/10.1109/ACCESS.2025.3585774>
- [7] Wadud, A., Golpayegani, F., Afraz, N.: xApp-Level Conflict Mitigation in O-RAN, a Mobility-Driven Energy-Saving Case (2025)
- [8] Polese, M., Bonati, L., D’Oro, S., Basagni, S., Melodia, T.: ColO-RAN: Developing machine learning-based xapps for open ran closed-loop control on programmable experimental platforms. *IEEE Transactions on Mobile Computing* (2023) <https://doi.org/10.1109/TMC.2022.3188013>
- [9] Bonati, L., Polese, M., D’Oro, S., Basagni, S., Melodia, T.: OpenRAN Gym: Ai/ml development, data collection, and testing for O-RAN on pawr platforms. *Computer Networks* **220**, 109502 (2023) <https://doi.org/10.1016/j.comnet.2022.109502>
- [10] Schmidt, R., Irazabal, M., Nikaein, N.: FlexRIC: An sdk for next-generation sd-rans. In: *Proceedings of the 17th International Conference on Emerging Networking Experiments and Technologies*, pp. 411–425. Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3485983.3494870> . <https://doi.org/10.1145/3485983.3494870>
- [11] Navarro, J., Castro, A., Astudillo, C.A., Rattaro, C.: Demystifying O-RAN implementations under resource-constrained hardware: An experimental evaluation. *Annals of Telecommunications* (2026) <https://doi.org/10.1007/s12243-026-01181-w>
- [12] Yap, R.H.C., Xia, W., Wang, R.: Generalized arc consistency algorithms for table constraints: A summary of algorithmic ideas. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 13590–13597 (2020). <https://doi.org/10.1609/aaai.v34i09.7086>
- [13] Lecoutre, C.: STR2: Optimized simple tabular reduction for table constraints. *Constraints* **16**(4), 341–371 (2011) <https://doi.org/10.1007/s10601-011-9107-6>
- [14] Bessière, C.: Arc-consistency in dynamic constraint satisfaction problems. In: *Proceedings of the Ninth National Conference on Artificial Intelligence*, pp. 221–226 (1991). <https://www.lirmm.fr/~bessiere/Site/stock/aaai91.pdf>
- [15] Dev Gupta, S., Genç, B., O’Sullivan, B.: Explanation in constraint satisfaction: A survey. In: *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence* (2021). <https://doi.org/10.24963/ijcai.2021/601> . <https://www.ijcai.org/proceedings/2021/601>
- [16] Perron, L., Didier, F.: CP-SAT Solver. Google OR-Tools documentation. Accessed 14 July 2024 (2023). https://developers.google.com/optimization/cp/cp_solver
- [17] Perron, L., Didier, F., Gay, S.: The CP-SAT-LP solver. In: *29th International Conference on Principles and Practice of Constraint Programming. Leibniz International Proceedings in Informatics* (2023). <https://doi.org/10.4230/LIPIcs.CP.2023.3> . Using OR-Tools v9.11. <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CP.2023.3>